

课程笔记

[LLM Agents F25] Predictable Noise in LLM Benchmarks

基于 Sida Wang 授课内容整理

2026-04-02



视频作者/频道: Berkeley RDI

发布日期: 2025 (Fall)

视频时长: 44:04

视频链接: <https://www.youtube.com/watch?v=HV8pugcFV00>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 课程定位：为什么今天要讨论“可预测噪声”	3
1.1 从“分数崇拜”到“分数校准”	3
1.2 噪声为何在 LLM 时代被放大	3
1.3 本章小结	4
2 噪声分类法：把不确定性拆成可建模组件	4
2.1 三层噪声结构	4
2.2 “可预测”的含义	4
2.3 噪声与“可比性”	4
2.4 本章小结	5
3 统计建模：把 benchmark score 当作随机变量	5
3.1 最小数学框架	5
3.2 bootstrap 与置信区间	5
3.3 噪声可预测性的工程收益	6
3.4 本章小结	6
4 协议噪声：Prompt、Seed、Decode 为什么会改变结论	6
4.1 Prompt template 不是中立变量	6
4.2 Decode 设置与 variance	6
4.3 Judge 噪声：LLM-as-judge 的双刃剑	7
4.4 本章小结	7
5 数据与任务噪声：污染、难度漂移与分布错配	7
5.1 数据污染 (contamination)	7
5.2 难度漂移 (difficulty drift)	7
5.3 任务分布错配	8
5.4 本章小结	8
6 从单轮到 Agent：为什么 agent benchmark 噪声更难	8
6.1 轨迹级方差 (trajectory-level variance)	8
6.2 环境不确定性与评测一致性	8
6.3 评估指标需要从“accuracy”扩展到“reliability”	9
6.4 本章小结	10
7 落地建议：团队如何构建抗噪声评测流水线	10
7.1 一个可执行的评测 SOP	10
7.2 当资源有限时，优先做什么	11
7.3 本章小结	11
8 实战案例：如何判断“+0.6 分”到底算不算进步	11
8.1 案例设定	11
8.2 从总体分数转向分桶结论	11

8.3	把显著性检验写进发布流程	12
8.4	本章小结	13
9	Agent 评测日志归因：从轨迹错误到系统改进	13
9.1	为什么必须看 trajectory log	13
9.2	错误归因表：把失败模式量化	13
9.3	一个简化的日志分析脚本	14
9.4	本章小结	15
10	组织治理：如何防止 benchmark 被“刷分工程”劫持	16
10.1	评测治理不只是技术问题	16
10.2	抗游戏化（anti-gaming）机制	16
10.3	面向 2026 的评测系统形态	16
10.4	本章小结	17
11	统计实操补充：如何设定“可发布”的显著性门槛	17
11.1	从 effect size 到 MDE（Minimum Detectable Effect）	17
11.2	重复运行预算：什么时候 5 次够，什么时候远远不够	18
11.3	多指标冲突：如何从“单分数”过渡到“发布门禁”	18
11.4	本章小结	18
12	附录：一份可直接复用的 Benchmark 报告模板	19
12.1	报告模板字段	19
12.2	发布前检查清单	19
12.3	本章小结	20
13	总结与延伸	20
13.1	全讲结论汇总	20
13.2	一句话复盘	21
13.3	拓展阅读	21

1 课程定位：为什么今天要讨论“可预测噪声”

1.1 从“分数崇拜”到“分数校准”

Sida Wang 这讲的起点很直接：当前社区对 benchmark 的使用方式，经常默认“分数差异 = 能力差异”。在小规模任务上这个假设有时成立，但在大模型和 agentic workload 下，分数本身已经变成高噪声统计量。也就是说，榜单并非无价值，而是必须先回答“这个分数的误差有多大”、“这个差异是否统计显著”。

本讲核心命题

Benchmark score 不是确定值，而是随机变量。只有把它放回概率与统计框架，排行榜才有可解释性。

1.2 噪声为何在 LLM 时代被放大

传统监督学习评测里，输入固定、输出空间较小、评分规则清晰，因此噪声通常主要来自采样误差。LLM 评测不同：模型输出自由度更高，prompt 模板选择空间更大，解码策略和 judge pipeline 都会注入额外方差。尤其在 open-ended 任务和 agent benchmark 中，轨迹级交互使 variance 成倍上升。

LLM benchmark 噪声放大的三个结构性原因

- **生成随机性**：temperature、top-p、sampling seed 会改变输出分布。
- **协议随机性**：prompt wording、system instruction、few-shot selection 会改写任务难度。
- **评审随机性**：LLM-as-judge、人评一致性、rubric 模糊性都会造成测量噪声。

Test set sizes are now much smaller than MNIST

Deep learning was measured and validated by MNIST and ImageNet
Testset sizes: MNIST: **10k**, ImageNet: **100k**
LLM multiple choice: MMLU ~10k

Generative test-based eval (generate a short program)
HumanEval: 164, MBPP+: 378, CruxEval: 800, DS-1000: 1000, DMC: 165, LiveCodeBench ~1000, Alder: 225

Agent Evals (often need 100k+ tokens, often hours)
SweBench: 2.3k, -Verified: 500, -lite: 300, -multimodal: 517
T-Bench: 80

图 1: 课程开场：从研究背景过渡到 benchmark noise 的问题定义¹

¹ 视频画面时间区间：00:03:20–00:03:35。

1.3 本章小结

这讲不是反对 benchmark，而是反对把 noisy metric 当作 deterministic truth。后续所有技术点都围绕同一个目标：把“分数”变成“可校准、可比较、可复现实验信号”。

2 噪声分类法：把不确定性拆成可建模组件

2.1 三层噪声结构

Sida 把噪声源拆为可以落地操作的几层：task/data 层、inference/protocol 层、evaluation/judge 层。这种分层的价值在于，它把“噪声”从模糊概念变成可诊断对象。团队在调研 benchmark 波动时，常见失败是把所有误差都归因到模型；分层框架要求先定位误差来自哪一层。

噪声层	典型来源	可控手段
Task/Data	数据污染、难度漂移、样本选择偏差	数据去污染、难度分桶、分布追踪
Inference/Protocol	prompt 模板、解码策略、上下文拼接方式	协议固定、seed 控制、A/B 模板对照
Evaluation/Judge	rubric 模糊、judge 偏置、评审一致性不足	双评审、仲裁机制、置信区间汇报

表 1: 可预测噪声的分层拆解框架

2.2 “可预测”的含义

可预测不等于可消除。实际系统里，很多噪声不会消失，但可以被估计、被约束、被报告。比如当你知道某个 benchmark 在固定配置下的 run-to-run standard deviation 是 0.8，那么 0.3 的改进就不应被当作“胜出”。这个思想本质是 measurement science，而不是 leaderboard engineering。

错误的报告方式

只报单点分数，不报 variance；只报最佳 run，不报 run distribution；只报总体平均，不报分桶结果。

2.3 噪声与“可比性”

社区经常做 cross-paper 对比，但如果协议不同、模板不同、judge 不同，分数本身不可比。Sida 的强调点是：先建立 comparability，再谈 superiority。否则论文 A 比论文 B 高 1 分，可能只是评测协议不同，而不是模型变强。

排行榜比较的常见误区

当两个系统不共享同一评测协议时，排序信息的可信度会显著下降。盲目比较只会制造伪进展。

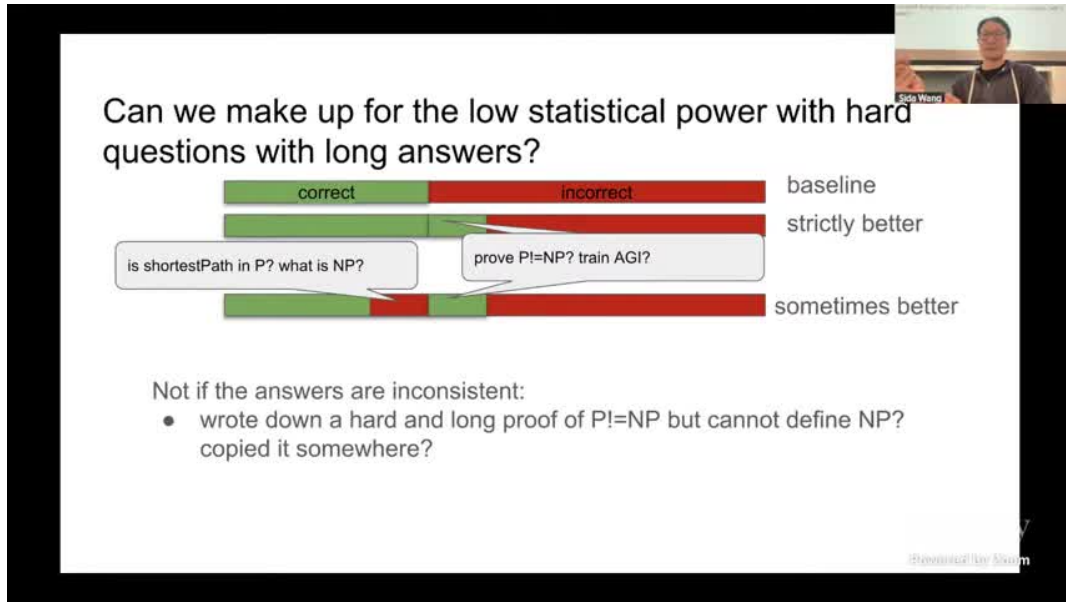


图 2: 课程中段: 噪声分类与评测设计关联关系²

2.4 本章小结

可预测噪声的关键不是“把噪声归零”，而是建立统一分层框架，让每个分数都能被解释。分层越清晰，后续统计估计和协议治理越容易落地。

3 统计建模：把 benchmark score 当作随机变量

3.1 最小数学框架

令某任务上的观测分数为 S ，理想能力为 μ ，噪声为 ϵ ，则

$$S = \mu + \epsilon, \quad \mathbb{E}[\epsilon] = 0, \quad \text{Var}(\epsilon) = \sigma^2.$$

如果只报告 S 而不报告 σ ，我们无法判断“改进”是否真实。进一步对多次运行 $S_1, \dots, S_n$ 取均值 $\bar{S}$ ，其方差缩放为 σ^2/n ，这解释了为什么多次重复评测是必要步骤。

为什么单次跑分不够

单次跑分只给你一个 realization，无法告诉你 score distribution。没有分布信息，就无法做显著性判断。

3.2 bootstrap 与置信区间

课程里反复提到 bootstrap 的价值：在样本规模有限、分布未知时，bootstrap 提供了实际可操作的 uncertainty estimate。对于 benchmark reporting，可以使用 percentile CI（如 95% CI）作为标准报告项。

²视频画面时间区间：00:11:40–00:11:58。

```

1 scores = run_eval_k_times(model, benchmark, k=20)
2 mean_score = np.mean(scores)
3 ci_low, ci_high = bootstrap_ci(scores, confidence=0.95)
4 report(mean_score, ci_low, ci_high)

```

Listing 1: Benchmark 置信区间计算的最小伪代码

报告规范建议

每个主结果至少包含：mean、std、95% CI、重复次数 k 、协议版本号。

3.3 噪声可预测性的工程收益

一旦噪声被量化，决策就会变得更理性。比如模型改进 pipeline 中，可以把“是否上线”从单分阈值改成“显著改进阈值”：只有当新模型相对旧模型在关键子集上显著优于阈值，才进入下一阶段。这会显著减少“看起来进步，线上退化”的事故。

未校准噪声会误导训练方向

如果评测噪声被误当作能力增益，训练团队会被带向错误目标，浪费计算预算和研究周期。

3.4 本章小结

这讲最重要的统计思想很朴素：分数必须和不确定性一起报告。bootstrap、重复评测和显著性检验不是锦上添花，而是 benchmark 能被信任的最低门槛。

4 协议噪声：Prompt、Seed、Decode 为什么会改变结论

4.1 Prompt template 不是中立变量

LLM 任务里，prompt 经常携带隐含 inductive bias。即便任务语义相同，不同模板也可能导致明显分差。Sida 在讲中强调，如果论文结论依赖某一个模板，必须检查它在模板扰动下是否稳定。

协议敏感性测试 (Protocol Sensitivity Test)

固定模型与数据，仅改变模板/解码参数，观察分数变化范围。若变化过大，说明结论脆弱。

4.2 Decode 设置与 variance

许多 benchmark 默认温度为 0，但在 agentic workload 中，模型通常无法完全 deterministic。此时 seed 和 decode policy 的影响会累积到轨迹层。课程里建议把 decode policy 当作评测协议的一部分进行 versioning，而不是隐含默认值。

协议版本管理建议

- 固定并公开: temperature、top-p、max tokens、stop criteria。
- 记录并公开: evaluation harness commit hash。
- 对关键结果做 seed sweep, 至少报告 3-5 个随机种子的分布。

4.3 Judge 噪声: LLM-as-judge 的双刃剑

LLM judge 能快速扩展评测规模, 但会引入 judge drift、position bias、verbosity bias 等问题。Sida 的态度是“可用, 但要校准”。常见做法包括双 judge 交叉验证、人评抽检、以及对评审 rubric 进行对抗测试。

Judge 过拟合风险

当模型针对 judge 偏好优化 (而非任务真实目标) 时, 会出现“会得分但不会做事”的现象。

4.4 本章小结

协议层噪声是最容易被忽略、也最容易被治理的一层。只要建立版本化协议、重复评测和 judge 校准流程, 很多“神秘波动”都能被解释为协议差异, 而非能力突变。

5 数据与任务噪声: 污染、难度漂移与分布错配

5.1 数据污染 (contamination)

模型训练语料与 benchmark 测试集重叠, 会显著高估泛化能力。课程中给出的核心观点是: 污染检测不应只做 exact match, 还应关注 paraphrase 和近重复。

污染治理的三层方案

1. 静态去重: n-gram / MinHash / embedding 近邻过滤。
2. 过程审计: 训练数据版本化与可追踪清单。
3. 评测防御: 动态更新测试集或构造 holdout private split。

5.2 难度漂移 (difficulty drift)

随着模型能力提升, 旧 benchmark 可能迅速饱和。此时分数增益压缩, 噪声占比上升, 导致“谁第一”变得更多取决于方差而不是能力差。Sida 建议对 benchmark 进行 difficulty stratification (按难度分桶) 并持续更新 hard split。

难度分桶的收益

分桶后可以区分两类改进：

- 在 easy split 提升：可能只是模板适配或表层优化。
- 在 hard split 提升：更可能代表真实能力增长。

5.3 任务分布错配

公开 benchmark 的任务分布往往与真实应用分布不同。对于 agent 系统，这种错配更严重，因为线上任务包含权限约束、工具失败、长上下文污染和多轮中断等现实因素。课程强调，benchmarks 应和 production telemetry 联动，而不是孤立存在。

只优化公开基准会产生局部最优

模型可能在 benchmark 上显著提升，但在真实任务链路里因为异常恢复和工具鲁棒性不足而退化。

5.4 本章小结

数据与任务层噪声决定了 benchmark “测到的到底是不是你想测的能力”。去污染、难度分桶和分布对齐是提升评测信度的三条主线。

6 从单轮到 Agent：为什么 agent benchmark 噪声更难

6.1 轨迹级方差 (trajectory-level variance)

在 agent benchmark 中，分数不仅受最终答案影响，还受中间决策路径影响。一个早期小错误会在后续工具调用中放大，形成高方差尾部事件 (long-tail failure)。这使得“同一模型、同一任务”在多次运行中的结果分布更宽。

Agent eval 的本质变化

从“静态样本打分”转向“动态轨迹打分”。评测对象不再是单输出，而是决策过程。

6.2 环境不确定性与评测一致性

Agent 任务依赖外部环境：API 延迟、网页变化、工具可用性、文件状态都可能改变结果。因此课程强调要把 environment versioning 当成 benchmark 协议的一部分，必要时使用 sandbox replay 或 deterministic simulator。

Agent benchmark 的最小可复现要素

- environment snapshot 版本号;
- tool API mock/real 的开关与记录;
- timeout 与 retry 策略;
- trajectory log 与 error taxonomy。

6.3 评估指标需要从 “accuracy” 扩展到 “reliability”

Sida 讲中指出, 对于 agent 系统, 仅看 success rate 不够。至少还应报告 recovery rate、invalid action ratio、tool efficiency、time-to-success 等指标。这样才能区分 “偶然成功” 与 “稳定可用”。

只看最终成功率会掩盖系统退化

两个系统可能成功率接近, 但一个需要大量无效工具调用和回退, 另一个路径更短更稳。若只看 success rate, 优化方向会偏离真实用户体验。

Paired comparison. $A(x)$ and $B(x)$ are correlated, and in an actual benchmark, the question x is used for both $A(x)$ and $B(x)$. To be more powerful for free, we can use the paired variance

$$\text{Var}_x[A(x) - B(x)] = \text{Var}_x[A] + \text{Var}_x[B] - 2\text{Cov}_x[A(x), B(x)]. \quad (4)$$

The Cov term may potentially reduce the paired variance to 0 when A and B are perfectly correlated.

Ask if this result is likely just due to chance

Once we have the standard error, we obtain the z-score $z = \frac{\bar{A} - \bar{B}}{\text{SE Var}[A - B]}$ which follows a standard normal distribution. For example, $\Pr[|z| > 1] = 0.32$ is very weak, $\Pr[|z| > 5] < 10^{-6}$ is beyond doubt, and $\Pr[|z| > 1.96] = 0.05$ for a p -value of 5% is a conventional and reasonable standard.

图 3: 课程后半段: 从单次分数转向分布与方差建模³

³视频画面时间区间: 00:20:30–00:20:46。

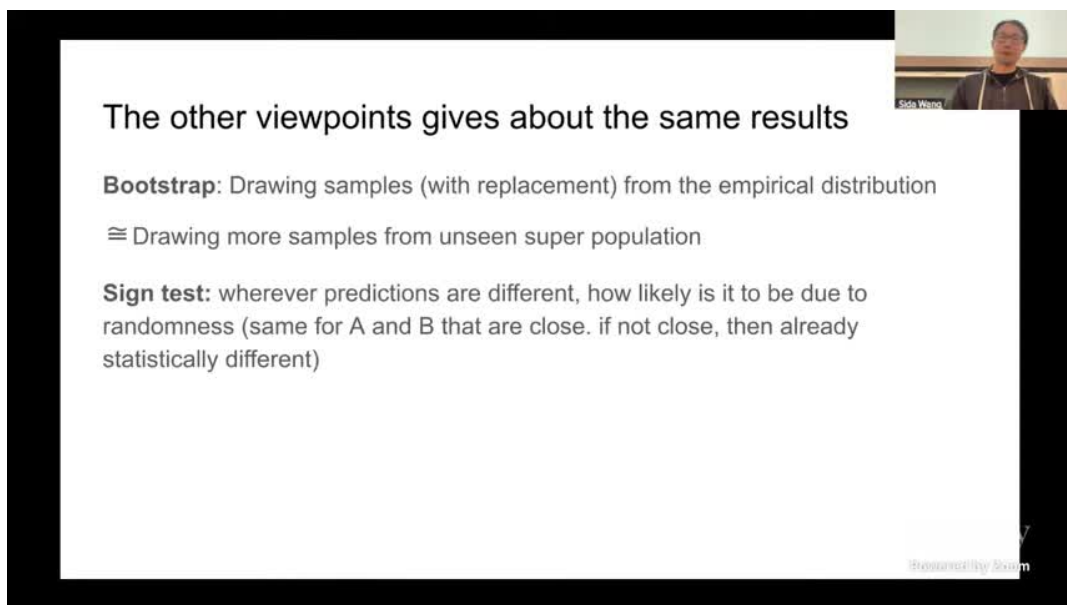


图 4: 统计方法部分：bootstrap 与显著性报告成为评测标准件⁴

6.4 本章小结

Agent benchmark 的难点不在“题更难”，而在“系统变量更多”。评测若不显式建模环境与轨迹噪声，就会把系统随机波动误判为模型能力变化。

7 落地建议：团队如何构建抗噪声评测流水线

7.1 一个可执行的评测 SOP

结合本讲内容，可以整理出一个可落地的评测 SOP：先定义任务层级和关键指标，再固定协议版本，然后做重复评测并报告置信区间，最后再做跨模板和跨环境鲁棒性验证。这个顺序能避免“先看排行榜再补解释”的被动流程。

阶段	动作	交付物
1	任务定义与分桶	task schema、难度分桶、目标指标
2	协议冻结	prompt 模板、decode 参数、judge 配置版本
3	重复评测	mean/std/CI、seed sweep 结果
4	鲁棒性检查	模板扰动、环境扰动、judge 一致性报告
5	上线判定	显著性门槛、回归报警规则、回滚策略

表 2: 抗噪声 benchmark 流水线建议

⁴视频画面时间区间：00:30:10–00:30:25。

7.2 当资源有限时，优先做什么

课程里一个务实观点是：很多团队不是不知道统计方法，而是没有预算全做。若只能选几件事，优先级建议是：重复评测（k 次运行）> 协议版本固定 > 最小置信区间报告 > 难度分桶。这四项能用最小成本换来最大可解释性提升。

低成本高收益实践包

- 同一配置至少跑 5 次并报均值与标准差；
- 每次发布写明协议版本与 seed；
- 对最关键 20% 样本做人评抽检；
- 对 hard split 单独汇报，不与总体平均混报。

7.3 本章小结

抗噪声评测不是大厂特权，而是一套可分层实施的方法。即使资源有限，也可以通过重复评测、协议冻结和最小统计报告显著提高结论可信度。

8 实战案例：如何判断“+0.6 分”到底算不算进步

8.1 案例设定

为了把课程中的统计思想落到工程决策，我们构造一个典型场景：团队有两个版本，Model-A（基线）和 Model-B（候选），在同一 benchmark、同一协议下各跑 10 次。平均分看起来 Model-B 高 0.6 分，但 run-to-run 波动也明显。

模型	Mean	Std	Runs	95% CI
Model-A	72.4	1.1	10	[71.7, 73.1]
Model-B	73.0	1.2	10	[72.2, 73.8]

表 3: 单看均值似乎 Model-B 更强，但置信区间仍有重叠

课程强调的决策规则

当两个模型置信区间重叠明显时，不应直接下结论“B 胜过 A”。更稳妥做法是：增加重复次数、做 paired test、并在关键子集上复核。

8.2 从总体分数转向分桶结论

把样本按难度分桶后，我们可能看到完全不同的结论：Model-B 在 easy split 提升很大，但在 hard split 几乎无提升。若真实业务更接近 hard split，那么总体 +0.6 分并不代表真实价值。

难度分桶	Model-A	Model-B	差值 (B-A)
Easy	84.1	85.7	+1.6
Medium	71.5	72.0	+0.5
Hard	58.8	58.9	+0.1

表 4: 分桶后可见改进主要来自 easy split，而非 hard split

只报总体分数会导致错误上线

如果业务请求以 hard case 为主，盲目按总体分数上线，可能带来“线上体感无提升甚至退化”的结果。课程中把这类问题称为 benchmark interpretation failure。

8.3 把显著性检验写进发布流程

在团队流程上，建议把“统计显著性门槛”变成 release gate。候选模型只有在关键任务集上满足预设门槛（例如 p-value 与 effect size 同时达标）才可进入灰度。这让评测从“结果展示”升级为“质量控制”。

可执行的发布门槛示例

- 关键任务集上 $\Delta\text{score} \geq 0.8$;
- paired bootstrap 的 95% CI 下界 > 0 ;
- hard split 不得退化超过 0.2 分;
- 工具调用效率指标 (tokens / tool calls / latency) 不得恶化超过阈值。

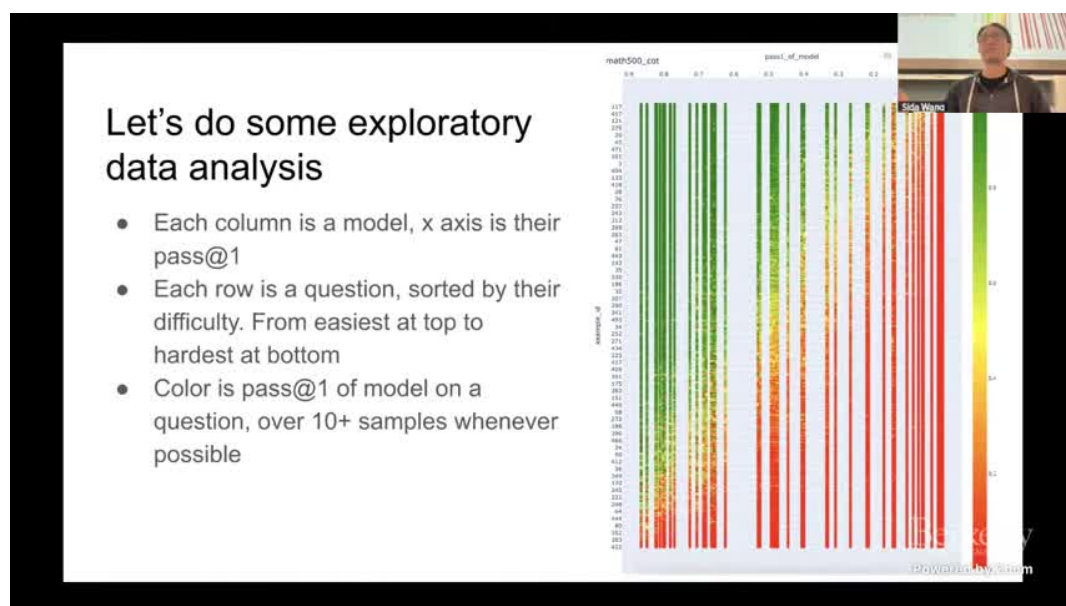


图 5: 案例讲解段：从“看均值”转向“看分布”⁵

⁵视频画面时间区间：00:06:40–00:06:55。

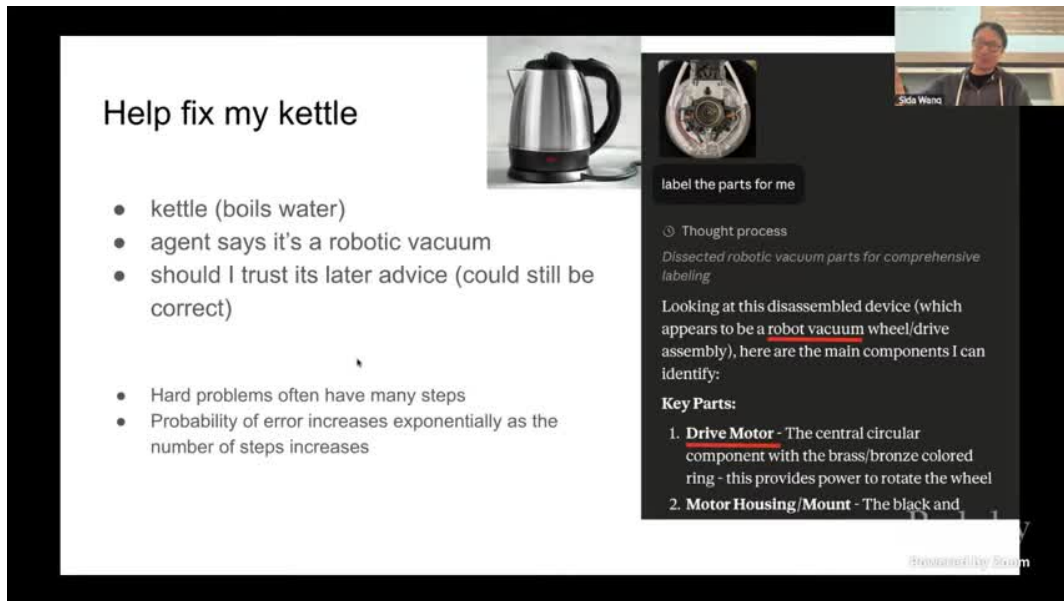


图 6: 协议设计讨论：比较结论依赖于统一评测协议⁶

8.4 本章小结

“+0.6 分”的意义取决于方差、分桶和业务分布。统计显著性不是学术装饰，而是模型发布的风险控制工具。

9 Agent 评测日志归因：从轨迹错误到系统改进

9.1 为什么必须看 trajectory log

在 agent benchmark 中，最终失败通常是多步错误叠加的结果。课程中反复强调，不看轨迹日志就很难判断“是模型 reasoning 错误”还是“工具接口失败”。这也是为什么 reliability 指标必须和 success rate 一起报告。

最小日志字段建议

- 每一步 action 的输入、输出、时间戳；
- 工具返回状态码与异常分类；
- 中间状态摘要（memory snapshot）；
- 终止原因（成功、超时、无效动作、评审拒绝）。

9.2 错误归因表：把失败模式量化

把失败模式结构化后，团队才知道应该优化模型、协议还是 infra。下面的表格示例体现了这一点：表面上 success rate 只有小幅提升，但 invalid action 与 timeout 大幅下降，说明系统稳定性改进显著。

⁶视频画面时间区间：00:16:10–00:16:26。

错误类型	Baseline	New	变化
Invalid Action	12.4%	8.1%	-4.3pp
Tool Timeout	9.7%	6.0%	-3.7pp
Wrong Plan	14.2%	13.0%	-1.2pp
Judge Rejection	6.8%	6.1%	-0.7pp

表 5: 轨迹级错误归因比单一 success rate 更有诊断价值

9.3 一个简化的日志分析脚本

为了让评测闭环更自动化，可以用轻量脚本聚合 trajectory 失败模式。关键不是脚本复杂度，而是全团队共享同一 taxonomy。

```

1 from collections import Counter
2
3 counter = Counter()
4 for traj in trajectories:
5     reason = traj["termination_reason"]
6     counter[reason] += 1
7
8 total = sum(counter.values())
9 for k, v in counter.items():
10    print(k, f"{v/total:.2%}")

```

Listing 2: 聚合 Agent 失败模式的最小示例

没有统一 taxonomy，日志会退化成噪声

如果每个子团队定义一套 termination reason，跨实验比较会迅速失效，最后又回到“只看总分”的旧问题。

Use the ability to draw independent samples?

This is a remarkable ability of the models, humans can't seem to do this. A common confusion is to consider this the only type of noise.

To capture this setting, we use ϵ for the seed generating the noise, which is independent of the question x , thus allowing the concise notation $\text{Var}_x[\mathbb{E}_\epsilon[A]] = \text{Var}_x[\mathbb{E}_{\epsilon|x}[A(x, \epsilon) | x]]$. As before, but now also averaging over the K samples for each question x , we consider the average score

$$\bar{A} = \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K A(x_i, \epsilon_{ij}) \approx \mathbb{E}_{x, \epsilon}[A(x, \epsilon)], \quad (5)$$

The estimated variance is then

$$\text{Var}_{x, \epsilon}[A(x, \epsilon)] \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K (A(x_i, \epsilon_{ij}) - \bar{A})^2. \quad (6)$$

Sida Wang

图 7: 课程中段: seed 与轨迹波动在 agent 场景中被进一步放大⁷

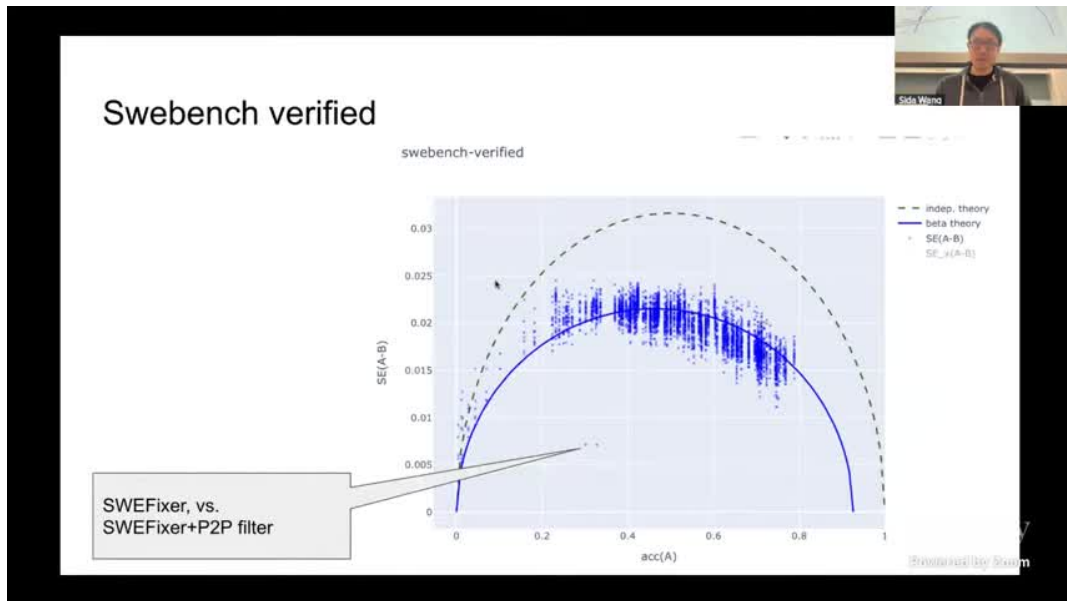


图 8: 后段讨论: agent eval 需要从结果导向转向过程导向⁸

9.4 本章小结

Agent 评测真正的增益来自“过程可解释”。轨迹日志、错误归因和统一 taxonomy 是把评测变成可优化系统的必要条件。

⁷ 视频画面时间区间: 00:24:40–00:24:58。

⁸ 视频画面时间区间: 00:34:20–00:34:38。

10 组织治理：如何防止 benchmark 被“刷分工程”劫持

10.1 评测治理不只是技术问题

Sida 在问答里提到，噪声问题最终会变成组织问题：如果团队激励只看 leaderboard，系统自然会朝“短期可刷分”方向演化。治理目标是让激励函数和真实能力一致，而不是让发布节奏被单一分数绑架。

评测治理的三条红线

1. 禁止只报 best run，不报完整 run distribution；
2. 禁止跨协议比较但仍给出确定性排序；
3. 禁止把私有调参结果当作通用能力结论对外发布。

10.2 抗游戏化（anti-gaming）机制

为了降低 benchmark gaming，可以采取 challenge split、协议轮换、隐藏测试集、以及周期性人工审计。课程强调，治理不是为了变慢，而是为了让“快”不以牺牲真实性为代价。

机制	解决的问题	代价
Hidden Test Split	防止公开集过拟合	复现成本上升
Protocol Rotation	防止模板投机	历史可比性下降
Human Audit	校准 judge 偏差	人力成本较高
Release Gating	把统计门槛写入流程	发布速度变慢

表 6: 治理机制是在“真实性”与“效率”间做工程折中

10.3 面向 2026 的评测系统形态

如果把本讲结论外推到下一阶段，团队需要的不只是 benchmark harness，而是 benchmark platform：可版本化协议、可追踪日志、可配置统计报告、可插拔 judge 和可审计发布流程。只有这样，benchmark 才能成为研发基础设施，而不是一次性演示工具。

没有治理，噪声会反向塑造研究方向

当团队把“噪声中奖”误当成“能力突破”，研究资源会被系统性错配，长期会拖慢真实进展。

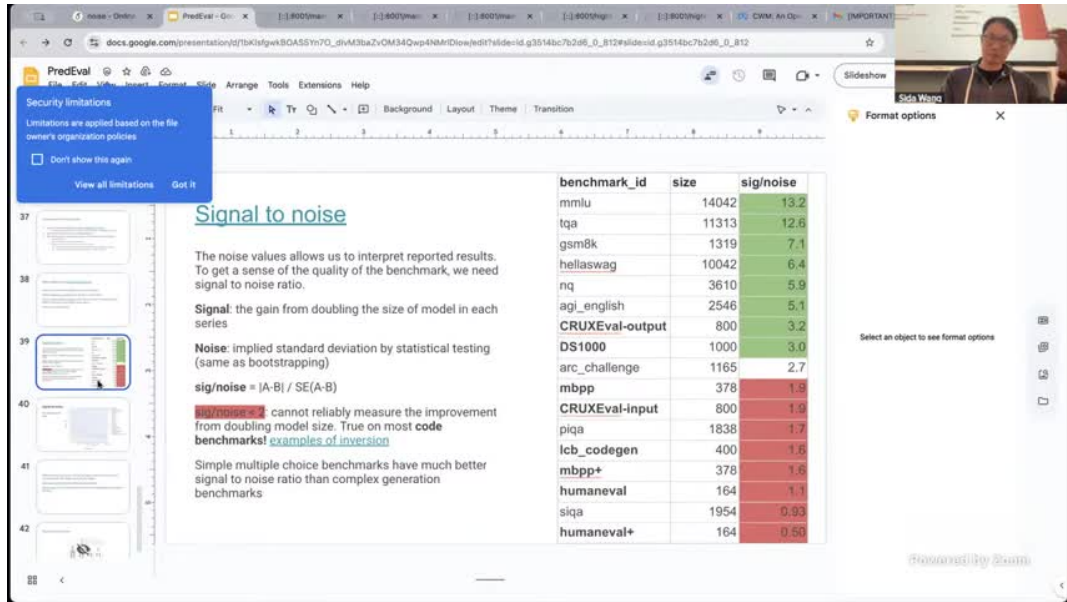


图 9: 问答部分：从技术细节延展到评测治理与组织实践⁹

10.4 本章小结

Benchmark 可靠性最终由组织机制保证。技术框架给出测量能力，治理机制保证测量结果不会被短期激励扭曲。

11 统计实操补充：如何设定“可发布”的显著性门槛

11.1 从 effect size 到 MDE (Minimum Detectable Effect)

课程反复强调“不要把微小分差直接解读成能力提升”。在工程实践中，可把这个原则落实为 MDE 机制：在发布前先声明“至少提升多少才算有效进步”，低于该阈值则进入观察区而非发布区。

MDE 的直观解释

MDE 不是统计上的真理，而是团队关于“业务上有意义提升”的共同约定。它把统计显著性和产品价值连接起来，避免为了追逐小数点后两位而过度优化 benchmark。

若把每次运行结果近似视为独立样本，均值比较的样本量可用如下近似式估计：

$$n \approx 2 \times \left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{\Delta/\sigma} \right)^2$$

其中 Δ 为目标检测差异 (MDE)， σ 为历史波动标准差， α 为显著性水平， β 为二类错误率。该式给出的不是绝对答案，但可快速评估“当前 run 数是否明显不足”。

常见误区：先跑完再找显著性

很多团队先跑几次，看到结果“似乎更好”才临时做显著性检验。这样会引入选择偏差。更稳健做法是提前写下评测计划：run 次数、停止条件、主指标与次指标、判定阈值。

⁹ 视频画面时间区间：00:42:10–00:42:28。

11.2 重复运行预算：什么时候 5 次够，什么时候远远不够

对于低噪声分类任务，5 次重复可能已经足够稳定；但对 agent benchmark，轨迹随机性和环境扰动会使方差显著上升，10–20 次运行才可能得到可解释区间。课程中的建议是基于历史方差做分层预算，而不是给所有任务统一 run 数。

任务类型	历史波动（示例）	建议 run 数	发布建议
Closed QA / MCQ	$\sigma \leq 0.3$	5–8	报告均值 + CI, 通常可快速发布
Open-ended generation	$0.3 < \sigma \leq 1.0$	8–12	强制双 judge 抽检
Agent workflow	$\sigma > 1.0$	12–20	必报轨迹级错误与环境版本

表 7: 按噪声强度配置运行预算，比固定 run policy 更稳健

可执行规则

当观测提升 $< \text{MDE}$ 或 95%CI 与基线显著重叠时，默认结论应为“暂无证据表明显著提升”，而非“模型退化/提升”的二元叙事。

11.3 多指标冲突：如何从“单分数”过渡到“发布门禁”

真实系统几乎总会出现指标冲突，例如 success rate 上升但 timeout 上升，或总体分数上升但 hard split 下降。课程建议把发布判定写成门禁规则（gating policy），例如：

- 主指标达到 MDE 且通过显著性门槛；
- 可靠性指标（timeout、invalid action）不得劣化超过阈值；
- hard split 不得出现统计显著退化；
- 若使用 LLM-as-judge，必须通过抽样人工复核。

为什么要“门禁化”

门禁化的核心价值是降低解释自由度。发布前就定义好规则，可以防止事后挑选有利指标，也让跨版本比较具有一致标准。

11.4 本章小结

统计实操的关键不是追求复杂检验，而是把“阈值、预算、门禁”前置定义。这样 benchmark 才能从研究展示工具升级为工程决策基础。

12 附录：一份可直接复用的 Benchmark 报告模板

12.1 报告模板字段

为了把课程思想直接落到团队协作，这里给出一份可复用模板。核心原则是“所有可影响结论的变量都要显式记录”，避免后续复现时出现信息缺失。

字段	示例	说明
Model Version Benchmark Ver- sion	model_2026_04_rc2 bench_v3.4	必须能唯一定位权重与配置 任务集与样本切分版本号
Prompt Protocol	proto_2026_04_a	system prompt + template + few-shot 策略
Decode Config	temp=0.2, top_p=0.95	生成参数是评测协议组成部 分
Judge Config	judge_gpt4.1_rubric_v2	若有 LLM judge, 需记录模 型和 rubric
Runs / Seeds	10 runs, seeds=[1..10]	无重复运行的结果不应进入 最终报告
Main Metric	success_rate=73.0	总体分数
Uncertainty	std=1.2, 95%CI=[72.2,73.8]	至少报告一种置信区间
Hard Split	58.9 (+0.1)	避免总体分数掩盖难样本退 化
Reliability Met- rics	invalid_action, time- out, recovery	Agent 任务必须补充可靠性 指标
Environment Ver- sion	env_snapshot_2026_04	保证任务环境可复现
Known Limita- tions	judge bias risk, sample drift	主动披露风险与限制

12.2 发布前检查清单

Release Checklist

1. 是否完成了不少于 5 次重复评测并保存全部原始日志？
2. 是否确认协议版本、judge 版本和环境版本均已冻结？
3. 是否报告了 hard split 与关键业务子集表现？
4. 是否有人评抽检来校准自动评审偏差？
5. 是否有回归告警阈值与回滚预案？

为什么这份模板有用

它把课程中的统计原则、协议治理和工程可复现性收敛成一份统一文档格式。模板统一后，跨团队比较将基于同一信息面展开，能显著降低“因为记录不全导致的争议”。

12.3 本章小结

评测体系要可持续，必须让“报告结构”标准化。模板和清单看似琐碎，但它们决定了 benchmark 结论能否被长期复用。

13 总结与延伸

13.1 全讲结论汇总

问题	课程结论	实践含义
为什么分数会波动？	分数本质是随机变量，噪声可预测但不可忽略	必须报告方差与置信区间
噪声来自哪里？	数据层、协议层、评审层共同作用	先分层诊断，再做治理
如何比较模型？	先保证 comparability，再谈 superiority	不同协议下分数不可直接排序
Agent 评测有什么不同？	轨迹级方差与环境噪声显著更高	需要环境版本化与可靠性指标
如何在团队落地？	建立抗噪声评测 SOP	从重复评测和协议冻结开始

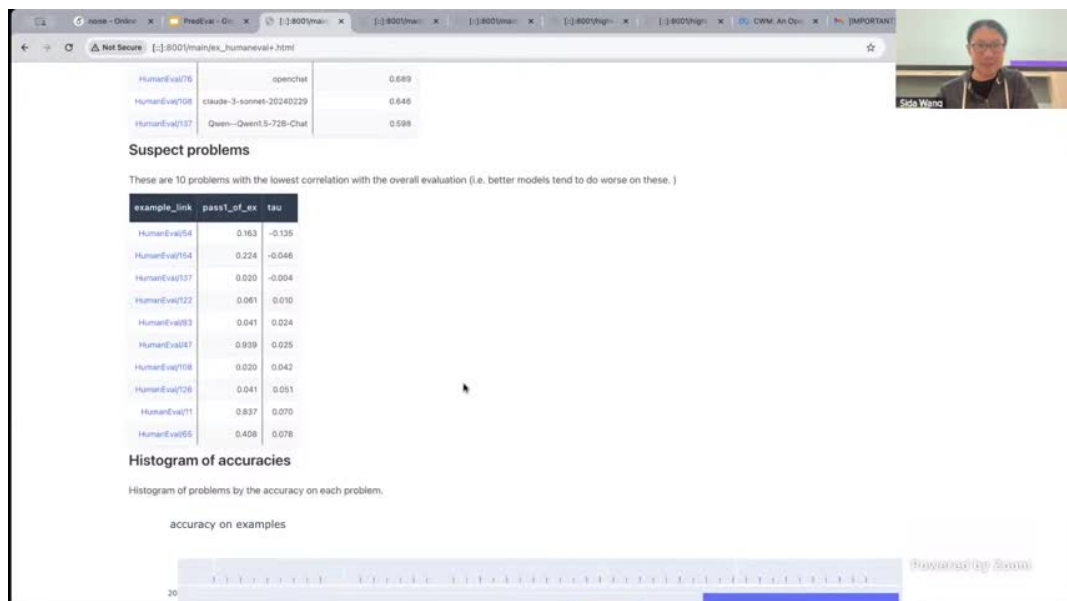


图 10: 课程收束：把 benchmark 从“榜单工具”升级为“测量系统”¹⁰

13.2 一句话复盘

Takeaway

如果不对 noise 建模，benchmark 会奖励偶然性；对 noise 建模后，benchmark 才会奖励真实进展。

13.3 拓展阅读

- Percy Liang et al., HELM (Holistic Evaluation of Language Models) 相关论文与报告
- “Beyond the Imitation Game” (LM 评测框架讨论)
- 统计学习中的 bootstrap 与 uncertainty quantification 经典教材
- 针对 LLM-as-judge 的 bias / consistency 研究工作
- Agent benchmark (WebArena / SWE-bench / BrowserGym 类) 中的复现与评测协议文档

¹⁰视频画面时间区间：00:40:20–00:40:36。